

Experiment 12: Counting Inversions

Aim

To implement a Python program to **count the number of inversions** in an array using the **Merge Sort** technique.

Algorithm

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the array elements.

Step 4: Divide the array into two halves recursively.

Step 5: Count inversions in the left half.

Step 6: Count inversions in the right half.

Step 7: While merging the two halves:

- If the left element is smaller, copy it.
- Otherwise, copy the right element and count the remaining elements in the left half as inversions.

Step 8: Display the total number of inversions.

Step 9: Stop.

Source Code (Python)

```
main.py
1 # Counting Inversions using Merge Sort
2
3 def merge(arr):
4     if len(arr) <= 1:
5         return arr, 0
6
7     mid = len(arr) // 2
8     left, inv_left = merge(arr[:mid])
9     right, inv_right = merge(arr[mid:])
10
11     result = []
12     i = j = inv = 0
13
14     while i < len(left) and j < len(right):
15         if left[i] <= right[j]:
16             result.append(left[i])
17             i += 1
18         else:
19             result.append(right[j])
20             inv += len(left) - i
21             j += 1
22
23     result += left[i:]
24     result += right[j:]
25
26     return result, inv + inv_left + inv_right
27
28
29 n = int(input("Enter number of elements: "))
30
31 arr = []
32
33 print("Enter the elements:")
34 for i in range(n):
35     arr.append(int(input()))
36
37 sorted_arr, inversions = merge(arr)
38
39 print("Sorted array:", sorted_arr)
40 print("Number of inversions:", inversions)
```

Output

```
Output
Enter number of elements: 5
Enter the elements:
2
4
1
3
5
```

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$

Space Complexity

- **$O(n)$**

Result

The **Counting Inversions** program was implemented successfully using the **Merge Sort** technique in Python. The program correctly counts the number of inversions in the given array and displays the sorted array along with the total inversion count.